

SecARP: Scalable ARP spoofing mitigation with a Distributed SDN

Tabeeb Khandaker, Seokjoo Cho, Sairam Penumarthy

University of Illinois Urbana-Champaign

ABSTRACT

ARP spoofing is a perennial problem in computer security. The stateless and trusting nature of the ARP protocol allows bad actors to mimic other hosts in a network and engage in attacks like Man-in-the-Middle. As a result, solutions to this problem have also appeared. We present in this paper a distributed SDN controller solution which drastically reduces latency for packets in the network while still remaining effective at detecting and mitigating ARP spoofing attempts.

1 INTRODUCTION

Public access networks, such as malls, airports, or even city wide wifi represent one of the backbones of internet connectivity. Large cafes or work from home centers are environments where people bring their own devices and are reliant on the connection they provide. These high churn networks require a degree of freedom and flexibility, and implementing effective and efficient security alongside this can be challenging. One of these issues lie in the Address Resolution Protocol (ARP) which all devices utilize for Layer 2 network connection.

From a security standpoint, ARP has major limitations, but that was by design. When ARP was originally developed the intent was to create a communication protocol to allow the discovery of link layer, or MAC, addresses. Since security was not a design goal, the protocol lacks authentication and always accepts ARP responses, making it trivial for malicious actors to launch arp spoofing and cache poisoning attacks. These can serve as a first step to advanced Man in the middle attacks. This issue has been addressed in public networks, most commonly with client isolation which blocks all direct peer to peer communication. In enterprise networks, proprietary switch hardware implements Dynamic Arp Inspection. Yet, both these solutions come with downsides that pave the way for our third option.

Software Defined Networking (SDN) provides the surface for our solution, with its ability to implement security at programmable controllers. There has been prior work using SDNs to protect against ARP spoofing and cache poisoning, but they were centralized approaches which required each ARP packet to be routed through a single point in the network. As opposed to other solutions to ARP hardening, this

added too much control plane latency. Our approach utilizes a distributed software defined network to mitigate this issue.

In this paper we present a system to provide security and low latency for large networks. We build upon the concepts presented in Scalable Ethernet Architecture using Software Defined Networking (SEASDN) [4] and use Open Network Operating System (ONOS) to implement a scalable defense against ARP for large public networks. SEASDN provides a methodology for checking ARP traffic, with its implementation relying on reactive querying for checking validity. Our primary contribution was adding a caching mechanism to proactively capture true DHCP mappings.

2 BACKGROUND AND MOTIVATION

Address Resolution Protocol is the backbone to local IPv4 communication, but was not designed with security as a focus. It relies on mutual trust between hosts and expects cooperation and fair use. In the real world, this is of course not the case. The protocol is stateless and trusting, meaning an unsolicited ARP reply without a corresponding ARP request is still accepted by any host that sees it. A malicious actor can take advantage of this, posing as a different host on the network by claiming their IP address in an ARP reply. This will poison the victims ARP cache, and their intended direct communication could be sent to the attacker. This makes way for man in the middle attacks, denial of service, and session hijacking.

It is important to note that in Software Defined Networks, ARP is often handled differently by the SDN controller. ONOS attempts to reduce ARP broadcast messages using a method called Proxy ARP. In Proxy ARP, all ARP messages are intercepted at switches and forwarded to the controller. From there, the controller will respond to the request if it has learned the IP and MAC address pairing that was being searched for, or it will default to broadcasting the request. The controller can learn these pairings by snooping packet headers in network traffic, but this is vulnerable to the same ARP spoofing attacks as regular ARP caches. Our solution works alongside Proxy ARP, and ensures that the controller doesn't learn these false mappings.

Prior solutions to this issue include client isolation and dynamic arp inspection. Client is the more comparable method

for our use case, as it is often implemented in public networks. This blocks direct communication between clients. This is effective as a security measure, as there is no way to perform ARP spoofing when clients are isolated from each other. On the other hand, this can be described as a “dumb” solution to the problem, as it breaks legitimate peer to peer communication. Some examples of this can include Apple’s Airplay or Google’s Chromecast, which provide utility in public workspaces.

Dynamic Arp Inspection (DAI) is often used in enterprise environments. In this system, switches are responsible for validating ARP packets against a trusted IP and MAC binding table, which is generated with DHCP snooping. This is more similar in implementation to how we validate ARP packets, in the sense that they are captured and checked against a source of truth. While effective and robust, DAI relies on proprietary and expensive hardware in switches with processing power requirements to handle packet inspection. As a result of this processing time, DAI switches can introduce new attack vectors such as ARP flooding. In addition, this method is less feasible for large scale deployments due to the cost of the switches.

We believe that a gap exists within the current solutions for a large public network that can provide the security of DAI without requiring new switches and retains full network connectivity. To fill this space we propose a distributed, software defined network. It would validate ARP packets with a source of truth in the SDN control plane. Given prior work in using SDNs for ARP spoofing mitigation, we knew there were effective security measures, but they were lacking robustness in terms of latency and controller bottlenecks. Our motivation was to design a system that utilized authoritative DHCP IP to MAC bindings to validate ARP packets, and to reduce overhead by caching those bindings at the controller.

3 DESIGN

For our design, we took certain steps to achieve effective ARP spoofing mitigation while also increasing scalability and reducing latencies. First, we needed to find a method which detects and mitigates ARP spoofing attacks. We landed on SEASDN [4] for its lightweight but accurate nature. However, we saw room for improvement by distributing the SDN controller. This required changes to the ARP cache system to quickly and accurately update ground truths for individual SDN controllers without generating a lot of overhead in calls to the DHCP server. Once these changes and methods were conceptualized, we needed an architecture to build on. These steps are detailed by the following subsections.

3.1 SEASDN

To detect and mitigate ARP spoofing in a computer network, we needed to first find a method that was effective.

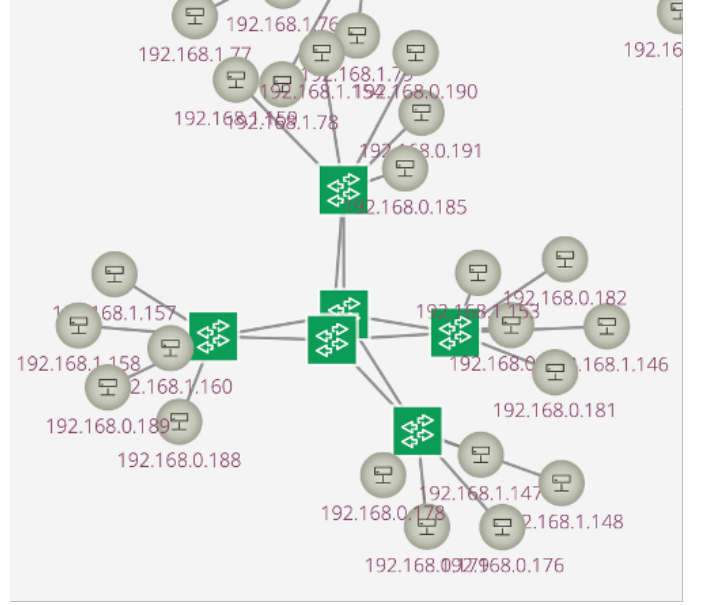


Figure 1: An example of our leaf-spine topology on one cluster, viewed through ONOS.

We decided to choose a method called SEASDN [4], which captured ARP packets in the network and compared IP and MAC address bindings in the packet to the DHCP server’s known mappings. Since the DHCP server initially assigns an IP address to hosts in the network, it effectively contains the ground truth of IP addresses among all hosts. This means that when a host lies about its IP, a simple check with the DHCP server’s mappings can verify if that IP is indeed what the MAC address in question was assigned by the DHCP server.

3.2 Improvements

We want to speed up this verification process while still maintaining the efficacy of the system. SEASDN utilizes a single SDN controller in its setup, which means all traffic in the network needs to go through a single point, leading to poor scalability and increasing latencies in busy networks. There is also an issue of a single point of failure. As such, we believe that distributing the system could lessen the load on the SEASDN controller. This led to new questions, such as how we were going to distribute the system, and how the ARP cache would need to change to facilitate such an action.

3.3 Architecture

Building and testing our solution required networking resources of some kind. We decided on using CloudLab instances. Our topologies would be formed by Mininet [7] Python scripts in each node generated by Cloudlab. The

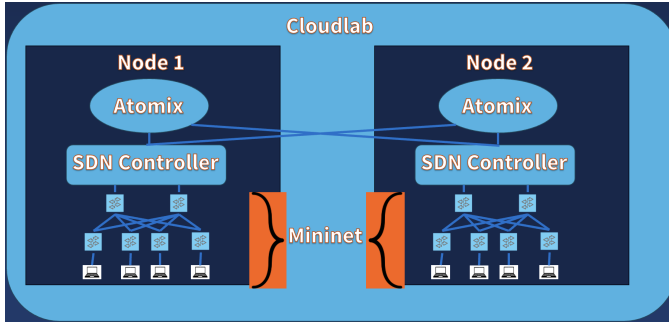


Figure 2: A simple 2-node and single-host-per-leaf example of our overall ONOS system.

topologies themselves followed a leaf-spine structure, displayed in Figure 1.

To distribute the system, we needed an SDN controller that was capable of it, and preferably open source. Open Network Operating System (ONOS) [1] provided just that, along with an easy to use GUI. Using Atomix instances as distributed stores, ONOS clusters could run as a distributed SDN. Figure 2 contains an example illustration of our architecture as a whole.

4 IMPLEMENTATION

Now that the bigger picture of the project is understood, our specific implementation details can be explained. We went through many forms for our project, since the way our system is structured drastically impacts the resulting performance and accuracy of our ARP spoof mitigator. This also meant a strong network architecture to test on needed to be produced. The complete source code is available at: <https://github.com/linuxlight/cs538-arp-spoof>.

4.1 Setting up our Distributed System

As mentioned in our design overview, we used a distributed ONOS network across small-lan Cloudlab nodes. This required using a distributed store called Atomix [3]. First, we implemented a skeleton distributed SDN with only two nodes, and Atomix attached to them as illustrated in Figure 2. SEASDN uses the network’s DHCP server to obtain ground truth IP-MAC bindings, so the next step was to set up a DHCP server in our network.

To do this, we explored two avenues. One was to create a custom DHCP server as a host connected to a leaf switch within the network. This ran into difficulties seeing as the SDN wouldn’t readily know where the DHCP server is. Plus, doing this on a single node would lead to extremely high latencies when pulling DHCP information from other nodes, and on multiple nodes, distribution would be too time-consuming to implement ourselves. As a result, we chose option two.

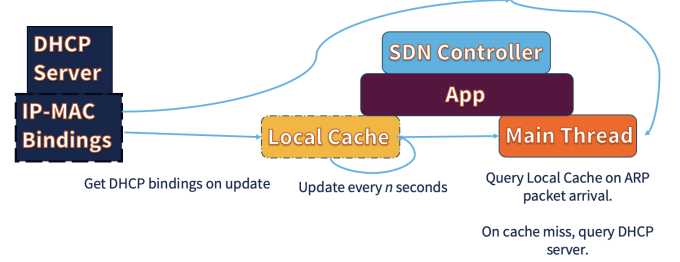


Figure 3: A diagram of our ONOS app’s workflow.

Our second option was to use ONOS’s DHCP server application. This worked out well for us, as the DHCP server’s distribution was handled by ONOS itself. We noticed pulling information from this server added a good amount of overhead though, at close to 30ms.

Once we understood how the DHCP server worked with the rest of our skeleton network, we moved on to how we would implement our Mininet topology. The leaf-spine architecture of Figure 1 helped balance traffic through the switches as they connected to their cluster’s SDN controller. It also kept the depth of the topology low while keeping the number of hosts high and allowing the hosts to all have the same distance to the SDN, making it easier for information to reach it quickly. We tested multiple node amounts, host numbers, and even Atomix numbers to verify scalability and check for further improvements.

4.2 Creating our Local Cache

The main contribution of our paper is the methodology we followed to improve latency accrued when detecting ARP spoofing. We set up our own custom ONOS app to implement our improvements, which would be active on each SDN controller.

By the very nature of a dynamic network, which gains and loses hosts at unpredictable times, a ground truth of the network needs to be constantly updated. Since our ground truth is the DHCP server’s IP-MAC bindings, we needed to verify a suspect ARP packet’s IP-MAC bindings with it whenever they appeared.

One possible solution to this is to pull from the DHCP server directly whenever a packet needs to be queried. This requires the `listMapping()` function in ONOS’s interfaces. However, `listMapping()` takes a relatively long time to run, adding approximately 30ms to an ARP packet’s RTT. It was clear that reducing our `listMapping()` calls would improve latency for ARP packets.

As a result, we decided to implement a background thread within each SDN’s app to update a local cache after some

Table 1: Hardware Specifications for CloudLab c220g5 Node

Component	Specification
CPU	2× Intel Xeon Silver 4114 (10-core) @ 2.20 GHz
RAM	192 GB ECC DDR4-2666
Disk	480 GB Intel DC S3500 6G SATA SSD
NIC	Dual-port Intel X520-DA2 10 Gb

time interval. The main thread of the app would continue to catch ARP packets and query the local, updated cache first. This meant that, on a cache hit, latency was reduced to very low times. Cache misses were affected by the standard overhead of a `listMapping()` call. However, this followed our expectations, as in practical cases, most packets will not be ARP spoofing, and cache hits will be more present. A diagram of this process is in Figure 3.

There are concerns of cache consistency if a local cache entry grows stale. When the main thread queries the stale entry before it can update from the DHCP server, then ARP spoofing packets may pass through. We ran tests to see how different local cache intervals affected our results. In practice, we feel as though this window is unavoidable to gain faster network traffic, and, over time, the chances of this happening are still quite small. Future work may aim to solve this issue while still benefiting from faster latency.

5 EVALUATION

5.1 Environmental Setup

We evaluate SecARP using a distributed testbed consisting of 8 physical nodes provisioned from CloudLab, described in Table 1. To simulate a large-scale network environment, we deploy a distributed Mininet instance across these machines, with each physical node hosting a subset of the virtual leaf-spine topology.

To benchmark the system, we execute an all-to-all concurrent stress test where a simultaneous arping process is initiated for every host pair for a 60-second duration. This provides a direct measurement of the ARP request-reply latency within the SDN control plane, allowing us to evaluate the controller’s ability to provide scalable, steady-state performance under continuous load across the entire topology.

5.2 Node Scalability

We evaluate the performance of SecARP against a baseline architecture modeled after SEASDN [4], where all distributed clusters communicate with a single centralized SDN controller without the benefit of a local cache. In contrast, our scheme utilizes a distributed SDN control plane across 8 physical nodes, optimized with a localized caching mechanism.

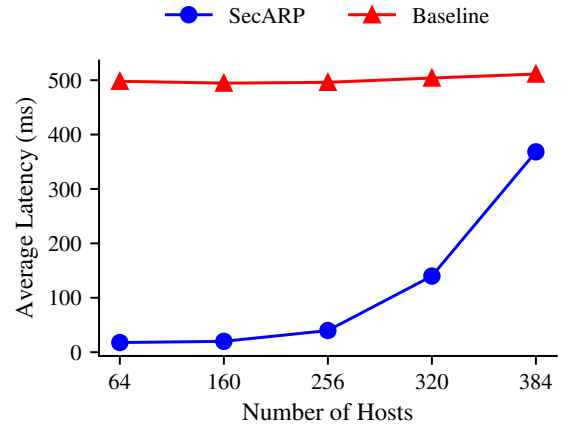

Figure 4: Average latency with various number of virtual mininet hosts of SecARP and baseline.

Figure 4 shows that our scheme maintains a consistent and stable latency even as the number of hosts increases. While a rise in latency is observed starting from 320 hosts, this behavior is attributed to hardware resource exhaustion during the benchmark. Each physical node, 20 cores with SMT, must manage 40 virtual hosts and thousands of concurrent arping processes. The observed performance degradation is attributed to host-level CPU saturation and process scheduling overhead, rather than a limitation of the SecARP scalability.

The Baseline scheme exhibits poor performance, maintaining a consistently high latency of approximately 500 ms regardless of host density. This is mainly attributed to the absence of a local cache, which forces the centralized controller to handle every ARP request as a packet-in event. This results in severe control plane congestion even at low host counts. We hypothesize that this saturation is caused by buffer queue limitations within the switch-to-controller communication channel. Under the heavy load of the all-to-all concurrent benchmark, these queues likely reach capacity, leading to tail-drop effects and significant processing delays as the ONOS controller struggles to clear the backlog.

5.3 Optimal Background Update Interval

We hoped to see how the background update interval affected our latencies. In Figure 5, we can see how, in general, increasing the interval affects the latency, decreasing over time. This is expected behavior as the update interval gets longer there should be less contention between security critical path and background thread.

We investigate how the background update interval influences system latency. As shown in Figure 5, increasing the interval generally results in a downward trend in latency. This is because a longer interval reduces the frequency of contention between the security-critical path and background

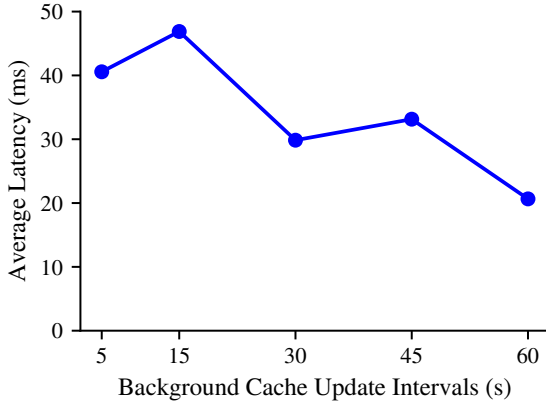


Figure 5: Average latency with various background cache update intervals.

update threads. Notably, the 30-second interval represents a sweet spot, where the system avoids the high context-switching overhead seen at shorter intervals, while providing enough time for the SDN controller to clear its packet buffers before the next update cycle begins.

While minor fluctuations occur at 15s and 45s, the overall trend confirms that reducing background thread frequency stabilizes performance by minimizing lock contention and CPU scheduling delays under heavy concurrent loads. However, this performance gain involves a compromise in cache accuracy. While the system retrieves data directly from the distributed DHCP ground truth upon a local cache miss to ensure eventual consistency, doing so from the critical path introduces significant latency due to distributed lookup overhead.

5.4 Distributed Store Scalability

As illustrated in Figure 6, latency increases significantly as the number of Atomix nodes grows, exhibiting a sharp spike at 6 nodes. This indicates that while increasing the node count can improve fault tolerance, it introduces substantial consensus overhead that negatively impacts performance.

This is a classic trade-off in distributed stores: fewer nodes provide lower latency due to reduced synchronization requirements but offer less system reliability. Conversely, if the Atomix cluster is too large, the communication overhead required to maintain the distributed DHCP map can lead to a performance bottleneck, especially under heavy traffic. Achieving an optimal balance between the number of SDN controllers and Atomix nodes is therefore crucial for performance.

5.5 Security

To evaluate the efficacy of SecARP in a live environment, we simulate various ARP-based threat vectors by modifying source IP and MAC address fields. We emulated two

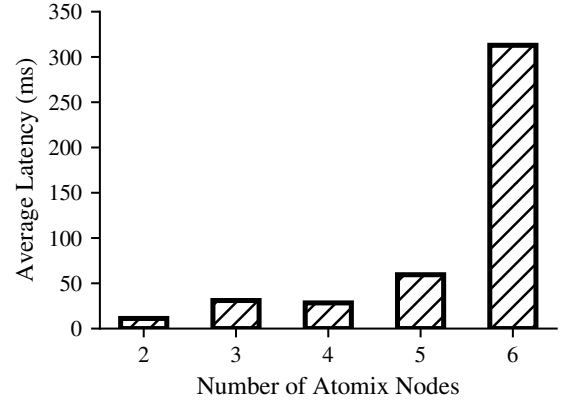


Figure 6: Average latency with various number of distributed store nodes.

primary attack scenarios: (1) a host claiming an unassigned IP address and (2) an attacker attempting to hijack an IP already assigned to a legitimate host. As demonstrated in Figure 7, SecARP correctly identifies these anomalies by cross-referencing incoming requests against the DHCP ground truth. When a mismatch is detected, the system generates a warning and proactively drops the malicious packets. Specifically, Figure 7a illustrates an unassigned IP claim, triggering ground truth verification, while Figure 7b confirms the detection of active spoofing. These results confirm that SecARP provides deterministic mitigation by falling back to the distributed store when local cache verification fails, effectively balancing high-speed performance with absolute security accuracy.

6 RELATED WORK

The threat of ARP spoofing is not new; it is a well documented issue with a variety of mitigation techniques. We categorized these existing solutions into hardware based, host-based, and SDN-based, and will discuss what they are and why our solution is better suited for large public networks.

As previously mentioned, the industry standard for hardening ARP is Dynamic Arp Inspection. DAI is implemented in high-end switches, and it intercepts ARP packets and validates them against a DHCP snooping binding table. While effective, DAI is a costly solution, especially for a large public network that would require many switches. In addition, DAI alone is still susceptible to denial of service attacks caused by flooding broadcast messages. [2]

A common defense in public networks is client isolation, which is also referred to as Public Secure Packet Forwarding. Client isolation is applied at the Access Point by blocking direct communication between wireless clients. It is highly effective at stopping ARP spoofing, as ARP replies will not


```
12:56:16.730 WARN [SecARP] Spoof detected: Claiming unassigned IP 30.0.0.1. MAC: DE:FB:5B:87:AA:3B
12:56:16.731 INFO [SecARP] LocalMap cache miss, calling dhcpService.listMapping
```

(a) Detection of unassigned IP claim.

```
12:54:08.768 WARN [SecARP] ARP SPOOFING DETECTED! IP 192.168.1.7 assigned to F6:75:8D:68:D6:13 is claimed by B6:FB:A5:E7:24:F6
12:54:09.724 WARN [SecARP] ARP SPOOFING DETECTED! IP 192.168.1.7 assigned to F6:75:8D:68:D6:13 is claimed by 9E:62:E9:EE:48:51
```

(b) Detection of active ARP spoofing for an assigned IP.

Figure 7: SecARP controller logs demonstrating real-time attack detection and mitigation.

get sent to other clients on the network. However, it is a blunt approach that blocks all peer to peer traffic, breaking modern applications like Chromecast and local file sharing. [6] We found this security/functionality tradeoff to be unacceptable.

The rise of Software Defined Networking has also led to new methods for handling ARP spoofing. Frameworks like SEASDN, which we based our validation method upon, suppress ARP requests and protect against spoofing. This method also protects against data plane flooding; it relies on the control plane to process every ARP request. With a singular controller, this can become a bottleneck for the network. [1] Other SDN based approaches attempt to detect spoofing by analyzing traffic patterns. These models are more probabilistic in nature, can produce inaccuracies, and come with higher overhead. [5]

Our system differs from these approaches by following a deterministic and proactive approach. Unlike DAI, our solution can run without proprietary hardware, such as with commodity servers and OpenFlow enabled switches. As opposed to reactive SDN approaches, we do not query the controller database for each ARP packet, as our local cache is able to provide accurate and low latency validation. Our solution also keeps full network functionality unlike client isolation.

7 LIMITATIONS

In the process of creating SecARP, we encountered numerous points of improvement. The detection and mitigation of ARP spoofing is still not a robust field, and though we contributed where we could in improving latencies, we made trade offs which could still be explored in the future.

The biggest limitation of SecARP is its allowance of cache inconsistency. As described before, the window at which a spoofing packet could fail to be mitigated is still very small, and requires multiple factors likely out of the control of the attacker to line up, like knowing an IP is changing before it actually does. The latency decreases significantly by allowing such a statistically low probability of error. However, this is still a chance. When working in the field of security, we want the utmost guaranties that our systems are safe. For our specific method, finding an optimal cache refresh period to reduce this chance while still maintaining low latency is

necessary. In the future, some new method of consistency between the DHCP server and local cache can be explored, which reduces latency while guaranteeing spoofing mitigation.

There is also the limitation of our overall network setup. We are effectively simulating our test network, which may not reflect real-life conditions. This means our latencies may not be true to how a network with hundreds of thousands of connections may act, though we have tried to stress test our system as much as possible. Moreover, we tested based on what we thought attackers would do when ARP spoofing. An attacker with a more unique method of ARP spoofing unknown to us may be able to effectively bypass our system, like a prolonged burst of ARP spoofing packets attempting to take advantage of our cache inconsistency limitation. There is simply not enough that we know of ARP spoof attackers to guarantee security here. A varied test set of attack methods could reveal more holes in our method, and possible paths forward.

We chose our setups and topologies after much thought, and even experiments in our evaluation, but there is still room for improvement. Future study could analyze the effectiveness of different Mininet topologies for ARP spoof mitigation. An optimal Atomix-to-ONOS SDN ratio could be studied as well, since this can affect latencies as shown in the evaluation. Even our DHCP setup may be suboptimal, since it is run by ONOS and could be customized to fit our use case.

8 CONCLUSION

SecARP, our ARP spoof mitigating distributed SDN system with a local background cache, scales well under standard network conditions. In our process of optimizing SecARP, we were also able to identify parameters which can impact the latency of packets in the network. Local cache update interval timing and the number of Atomix nodes used all impact that timing, and values can be chosen which work better than others.

There is still much to improve with this, ranging from removing cache inconsistencies to using more realistic data traffic. However, we believe SecARP is a promising step in the direction of effective, low latency, and lightweight ARP spoofing detection and mitigation.

ACKNOWLEDGEMENTS

We would like to thank Professor Francis for his feedback throughout this process, as those questions bore further developments and results to our project. We would also like to thank Cloudlab, as the administrators generously extended our network resource usage over multiple months of testing.

REFERENCES

- [1] Administrator. 2020. ONOS. <https://wiki.onosproject.org> Accessed: 2025-12-18.
- [2] Cisco Meraki. 2025. Dynamic ARP Inspection. https://documentation.meraki.com/Switching/MS_-_Switches/Operate_and_Maintain/How-Tos/Dynamic_ARP_Inspection Accessed: 2025-12-18.
- [3] Github. 2022. Atomix. <https://github.com/atomix/atomix-archive>
- [4] Naseela Jehan and Aneesh M. Haneef. 2015. Scalable Ethernet Architecture Using SDN by Suppressing Broadcast Traffic. *2015 Fifth International Conference on Advances in Computing and Communications (ICACC)* (2015), 24–27.
- [5] Huan Ma, Hao Ding, Yang Yang, Zhenqiang Mi, and Miao Zhang. 2015. SDN-Based ARP Attack Detection for Cloud Centers. In *2015 IEEE 12th Intl Conf on Ubiquitous Intelligence and Computing and 2015 IEEE 12th Intl Conf on Autonomic and Trusted Computing and 2015 IEEE 15th Intl Conf on Scalable Computing and Communications and Its Associated Workshops (UIC-ATC-ScalCom)*. 1049–1054. <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCCom-IoP.2015.195>
- [6] Dr. Timur Mirzoev and Stacey White. 2014. The Role of Client Isolation in Protecting Wi-Fi Users from ARP Spoofing Attacks. arXiv:1404.2172 [cs.NI] <https://arxiv.org/abs/1404.2172>
- [7] Web. 2022. Mininet. <https://mininet.org/>